

MARCOS DE DESARROLLO WEB

Guía de laboratorio de la semana 9

Persistencia con JPA Hibernate Spring Data JPA Flyway y MySQL

PRODUCTO DE LA SEMANA

TechLab conectado a MySQL mediante una capa de persistencia de solo lectura, con esquema versionado por Flyway, entidad JPA, repositorio, servicio transaccional y pruebas reproducibles.

Proyecto acumulativo TechLab

Propósito y alcance

En la semana 8, TechLab atendía solicitudes MVC y REST con una lista guardada en memoria. En esta práctica se sustituye únicamente esa fuente de datos por MySQL. Las rutas, plantillas y contratos HTTP se mantienen, de modo que el estudiante pueda observar cómo una arquitectura por capas permite cambiar la infraestructura sin reescribir la interfaz web.

Logro de aprendizaje

Al finalizar, el estudiante configura una fuente de datos MySQL, explica la relación entre ORM, JPA, Hibernate y Spring Data JPA, versiona el esquema con Flyway e implementa consultas de solo lectura sin exponer entidades de persistencia a la capa web.

Criterios observables

- Diferencia con precisión JPA, Hibernate, Spring Data JPA, JDBC y MySQL.
- Mantiene credenciales fuera del repositorio y valida la configuración antes de iniciar.
- Crea el esquema exclusivamente con migraciones Flyway ordenadas e inmutables.
- Mapea la tabla cursos con una entidad JPA y consulta mediante un repositorio.
- Conserva DTO, servicio y controladores con responsabilidades separadas.
- Demuestra el comportamiento con pruebas automáticas y una matriz manual.

Límite de la semana No se implementan formularios de alta, edición o eliminación. Esas operaciones corresponden a la semana 10. Tampoco se agregan relaciones entre entidades ni Bean Validation, que se desarrollarán en la semana 11.

Ruta de trabajo

Tabla 2 Secuencia sugerida para la sesión

Bloque	Actividad	Evidencia
1	Diagnóstico y copia segura	Línea base aprobada
2	Modelo conceptual y dependencias	Decisiones explicadas
3	MySQL y migraciones Flyway	Esquema versionado
4	Entidad repositorio mapper y servicio	Lecturas persistentes
5	Pruebas evidencias y reto	Entrega verificable

1 Preparar una línea base confiable

Trabaje sobre una copia o rama del proyecto de la semana 8. Primero compruebe que la versión sin base de datos todavía compila y pasa sus pruebas. Si la línea base falla, no agregue JPA hasta corregir el primer error reproducible.

Requisitos

Tabla 3 Lista de requisitos

Componente	Versión o condición	Comprobación
JDK	Java 25 LTS	java --version
Maven	Wrapper incluido en el proyecto	./mvnw --version
Docker	Engine o Desktop con Compose v2	docker compose version
Puertos	8080 para web y 3307 para MySQL	Verificar que estén libres
Proyecto	TechLab de la semana 8	./mvnw clean test

Puerta de entrada

1. Guarde o confirme todos los cambios de la semana 8.
2. Cree la rama week-09-persistence o una copia fechada del proyecto.
3. Ejecute `java --version`, el wrapper de Maven y las pruebas existentes.
4. Abra `/cursos`, `/cursos/resumen` y `/api/v1/cursos` antes de modificar el servicio.
5. Registre el identificador del commit o la fecha de la copia en el README.

Criterio de detención Si el proyecto base no alcanza BUILD SUCCESS o alguna ruta no responde, resuelva esa causa antes de continuar. Mezclar defectos heredados con errores de persistencia vuelve el diagnóstico innecesariamente difícil.

Estructura objetivo

Árbol del proyecto

```
techlab-web/
```

```

├── .env.example
├── .gitignore
├── compose.yaml
├── pom.xml
├── mvnw / mvnw.cmd / .mvn/wrapper/
├── src/
│   ├── main/
│   │   ├── java/pe/edu/utp/techlab/
│   │   │   ├── controller/ (sin cambios)
│   │   │   ├── dto/CourseDto.java
│   │   │   ├── entity/Course.java
│   │   │   ├── mapper/CourseMapper.java
│   │   │   ├── repository/CourseRepository.java
│   │   │   ├── service/CourseService.java
│   │   └── resources/
│   │       ├── application.properties
│   │       ├── db/migration/
│   │       │   ├── V1_create_cursos_table.sql
│   │       │   └── V2_seed_cursos.sql
│   │       ├── static/ (sin cambios)
│   │       └── templates/ (sin cambios)
│   └── test/java/pe/edu/utp/techlab/ (controller y persistence)

```

2 Comprender la capa de persistencia

La base de datos no se incorpora directamente al controlador. La solicitud sigue atravesando las mismas capas, pero el servicio delega la lectura en un repositorio y este utiliza JPA. Hibernate traduce la operación a SQL y el controlador recibe DTO, no entidades administradas.

Tabla 4 Responsabilidad de cada tecnología

Concepto	Responsabilidad	Aplicación en TechLab
MySQL	Almacenar tablas índices y filas	Base techlab y tabla cursos
JDBC	Protocolo Java de acceso relacional	Usado por la infraestructura
JPA	Especificar el mapeo objeto relacional	Anotaciones jakarta.persistence
Hibernate	Implementar JPA y generar SQL	Proveedor administrado por Spring Boot
Spring Data JPA	Crear repositorios a partir de interfaces	CourseRepository
Flyway	Versionar y ejecutar cambios de esquema	Migraciones V1 y V2

Flujo de una lectura

Tabla 5 Recorrido de los datos

Etaa	Componente	Entrada	Salida
1	CourseViewController	GET /cursos?q=boot	Invocación al servicio
2	CourseService	Criterio normalizado	Solicitud al repositorio
3	CourseRepository	Método derivado	Consulta JPA
4	Hibernate y JDBC	Modelo y conexión	SQL y filas
5	CourseMapper	Entidad Course	CourseDto
6	Thymeleaf o Jackson	DTO	HTML o JSON

Entidad y DTO

Tabla 6 Separación entre persistencia y representación

Aspecto	Entidad Course	CourseDto
Propósito	Representar persistencia	Representar salida de aplicación
Anotaciones	@Entity @Id @Column	Ninguna dependencia de JPA
Ciclo de vida	Administrado por Hibernate	Objeto inmutable ordinario
Exposición web	No debe serializarse	Sí puede llegar a MVC o REST
Cambio de esquema	Puede requerir migración	Puede evolucionar por contrato

Decisión arquitectónica spring.jpa.open-in-view=false obliga a resolver el acceso a datos dentro de la transacción del servicio. Esta guía lo adopta desde el inicio para evitar consultas ocultas durante el renderizado y mantener la capa web independiente.

Comprobación conceptual

Tabla 7 Clave breve de conceptos

Pregunta	Respuesta esperada
Qué elemento es una especificación	JPA
Qué elemento implementa JPA	Hibernate
Quién genera el repositorio	Spring Data JPA
Quién crea y versiona la tabla	Flyway
Por qué se conserva CourseDto	Para no filtrar el modelo de persistencia

3 Actualizar dependencias

Spring Boot administra las versiones compatibles de Hibernate, Spring Data, Flyway y el controlador JDBC. No agregue números de versión a dependencias individuales. Mantenga una sola versión explícita: la del parent de Spring Boot.

pom.xml parte 1 de 3

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>4.1.1</version>
    <relativePath/>
  </parent>

  <groupId>pe.edu.utp</groupId>
  <artifactId>techlab-web</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>techlab-web</name>
  <description>Laboratorio de persistencia de la semana 9</description>

  <properties>
    <java.version>25</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-webmvc</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-thymeleaf</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-actuator</artifactId>
    </dependency>
```

pom.xml parte 2 de 3

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-flyway</artifactId>
</dependency>
<dependency>
  <groupId>org.flywaydb</groupId>
  <artifactId>flyway-mysql</artifactId>
</dependency>
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
  <scope>runtime</scope>
  <optional>true</optional>
</dependency>
```

pom.xml parte 3 de 3

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webmvc-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
</build>
</project>
```

Tabla 8 Dependencias nuevas

Dependencia	Función	Criterio
starter-data-jpa	JPA Hibernate Spring ORM	Código de producción
starter-flyway	Integración de migraciones	Una única estrategia de esquema
flyway-mysql	Soporte específico de MySQL	Requerido para este motor
mysql-connector-j	Controlador JDBC	Solo tiempo de ejecución
starter-data-jpa-test	Pruebas de persistencia	Solo alcance test

Comprobación Ejecute `./mvnw dependency:tree` y confirme que existe una sola línea principal de Spring Framework, Hibernate y Flyway. No corrija conflictos añadiendo versiones al azar; primero identifique qué dependencia introdujo la duplicidad.

4 Provisionar MySQL de forma reproducible

La ruta recomendada utiliza Docker Compose porque todos los estudiantes obtienen la misma versión y configuración. Si ya dispone de MySQL 8.4, puede usar una instalación local, pero debe conservar el nombre de base, el usuario de aplicación y el conjunto de caracteres indicados.

Excluir secretos y artefactos

.gitignore

```
target/
.idea/
.vscode/
*.iml
.env
application-local.properties
```

Regla de seguridad El archivo `.env` real nunca se entrega ni se confirma en Git. Solo se comparte `.env.example` con nombres de variables y valores de muestra no sensibles.

.env.example

```
DB_USER=techlab_app
DB_PASSWORD=reemplazar_con_una_clave_local
MYSQL_ROOT_PASSWORD=reemplazar_con_otra_clave_local
```

1. Copie `.env.example` como `.env`.
2. Sustituya ambos valores de contraseña por claves locales distintas.
3. Compruebe con `git status` que `.env` no aparece entre los archivos por confirmar.

compose.yaml

```

services:
  mysql:
    image: mysql:8.4.12
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: techlab
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
    ports:
      - "3307:3306"
    volumes:
      - techlab_mysql_data:/var/lib/mysql
    healthcheck:
      test:
        - CMD-SHELL
        - >-
          mysqladmin ping -h localhost -uroot
          -p${MYSQL_ROOT_PASSWORD} --silent
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 30s

volumes:
  techlab_mysql_data:
    name: techlab_mysql_data

```

El puerto 3307 del equipo se conecta al 3306 del contenedor y reduce conflictos con instalaciones locales. El volumen con nombre conserva los datos cuando el contenedor se reinicia. El healthcheck informa cuándo MySQL acepta conexiones.

Iniciar y comprobar el servicio**Terminal**

```

docker compose config --quiet
docker compose up -d mysql
docker compose ps
docker compose logs mysql --tail=30

```

Resultado esperado docker compose ps debe mostrar el servicio mysql como running y, después del periodo de inicio, healthy. Use los logs solo para diagnóstico y no incluya capturas que revelen valores sensibles.

Conectar la aplicación**Tabla 9 Variables para el proceso Java**

Sistema	Comando previo a Maven
Linux macOS o Git Bash	export DB_USER='techlab_app' export DB_PASSWORD='valor_de_su_env'
Windows PowerShell	\$env:DB_USER='techlab_app' \$env:DB_PASSWORD='valor_de_su_env'

Las variables se definen en la misma terminal donde se ejecutará Maven. Docker Compose lee .env para crear MySQL; Java recibe las variables del proceso y no lee ese archivo automáticamente.

5 Configurar la fuente de datos

src/main/resources/application.properties

```
spring.application.name=techlab-web
server.port=8080

spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3307/techlab?serverTimezone=UTC}
spring.datasource.username=${DB_USER:techlab_app}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
spring.jpa.show-sql=false
spring.jpa.properties.hibernate.jdbc.time_zone=UTC

spring.flyway.enabled=true
spring.flyway.locations=classpath:db/migration
spring.flyway.validate-migration-naming=true

management.endpoints.web.exposure.include=health
management.endpoint.health.show-details=never

server.error.include-message=never
server.error.include-binding-errors=never
server.error.include-stacktrace=never
```

Tabla 10 Decisiones de configuración

Propiedad	Motivo
datasource.url	Define host puerto esquema y zona horaria
datasource.password	Exige DB_PASSWORD y falla si no existe
ddl-auto=validate	Compara el mapeo con el esquema sin modificarlo
open-in-view=false	Evita acceso tardío a datos desde la vista
flyway.locations	Establece la fuente única de cambios de esquema
error.include-*	Impide publicar mensajes internos y trazas

No use ddl-auto=update Esa opción modifica el esquema de manera implícita y no produce una historia revisable. Flyway crea y evoluciona el esquema; Hibernate únicamente valida que el mapeo coincida con él.

No combine inicializadores Al usar Flyway, no cree schema.sql ni data.sql. Una sola estrategia evita órdenes ambiguos, datos repetidos y diferencias entre equipos.

Secuencia de arranque

Tabla 11 Fallo temprano durante el arranque

Orden	Acción	Fallo que detecta
1	Construir DataSource	URL usuario o contraseña inválidos
2	Ejecutar Flyway	SQL inválido o migración alterada
3	Crear EntityManagerFactory	Entidad y tabla incompatibles
4	Crear repositorios y servicios	Beans o consultas inválidas
5	Abrir servidor web	Puerto ocupado o configuración web

6 Versionar el esquema con Flyway

Cada migración tiene un número y una descripción separados por dos guiones bajos. Flyway registra su checksum y ejecución en flyway_schema_history. Una migración aplicada no se edita: cualquier cambio posterior recibe un nuevo número.

src/main/resources/db/migration/V1_create_cursos_table.sql

```
CREATE TABLE cursos (
  id BIGINT NOT NULL AUTO_INCREMENT,
  titulo VARCHAR(120) NOT NULL,
  horas INT NOT NULL,
  CONSTRAINT pk_cursos PRIMARY KEY (id),
  CONSTRAINT uk_cursos_titulo UNIQUE (titulo),
  CONSTRAINT chk_cursos_horas CHECK (horas BETWEEN 1 AND 500)
) ENGINE=InnoDB
DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci;
```

La clave primaria identifica cada curso; el índice único evita títulos duplicados; CHECK protege el rango incluso si otro cliente escribe en la base. utf8mb4 conserva todo Unicode y la intercalación indicada ignora mayúsculas en comparaciones habituales.

src/main/resources/db/migration/V2_seed_cursos.sql

```
INSERT INTO cursos (id, titulo, horas) VALUES
(1, 'HTML y CSS', 12),
(2, 'Bootstrap', 16),
(3, 'Spring Boot', 20);

ALTER TABLE cursos AUTO_INCREMENT = 4;
```

Por qué dos archivos V1 describe estructura y V2 carga el catálogo inicial. Separarlos permite auditar responsabilidades y agregar datos o cambios posteriores sin reescribir la historia.

Primera ejecución

Terminal en Linux macOS Git Bash o Windows PowerShell

```
# Linux, macOS o Git Bash
export DB_USER='techlab_app' DB_PASSWORD='valor_de_su_env'
./mvnw spring-boot:run
# Windows PowerShell
$env:DB_USER='techlab_app'; $env:DB_PASSWORD='valor_de_su_env'
.\mvnw.cmd spring-boot:run
```

7 Mapear la entidad y crear el repositorio

src/main/java/pe/edu/utp/techlab/entity/Course.java

```
package pe.edu.utp.techlab.entity;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "cursos")
public class Course {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "titulo", nullable = false, length = 120)
    private String titulo;

    @Column(name = "horas", nullable = false)
    private int horas;

    protected Course() {
    }

    public Long getId() {
        return id;
    }

    public String getTitulo() {
        return titulo;
    }

    public int getHoras() {
        return horas;
    }
}
```

Tabla 12 Lectura de la entidad

Elemento	Significado
@Entity	La clase participa en el contexto de persistencia
@Table	Vincula la clase con la tabla cursos
@Id	Marca la identidad persistente
IDENTITY	MySQL genera el valor AUTO_INCREMENT
Constructor protected	Cumple JPA sin formar parte de la API pública
Sin setters públicos	Evita mutaciones que la semana aún no necesita

Convención No use @Data de Lombok en entidades. Los equals, hashCode y toString generados sin revisar pueden incorporar asociaciones perezosas, producir recursión o cambiar cuando se asigna el identificador.

src/main/java/pe/edu/utp/techlab/repository/CourseRepository.java

```

package pe.edu.utp.techlab.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;

import pe.edu.utp.techlab.entity.Course;

public interface CourseRepository extends JpaRepository<Course, Long> {

    List<Course> findAllByIdAsc();

    List<Course> findByTituloContainingIgnoreCaseOrderByIdAsc(
        String titulo
    );

    @Query("select sum(c.horas) from Course c")
    Long sumarHoras();
}

```

Spring Data analiza los nombres `findAllByIdAsc` y `findByTituloContainingIgnoreCaseOrderByIdAsc` para crear las consultas. La suma se expresa con JPQL: `Course` y `horas` son nombres del modelo Java, no de la tabla SQL. El resultado puede ser null cuando no existen filas, por eso el servicio aplica el valor cero.

Tabla 13 Contrato de lectura del repositorio

Método	Consulta lógica	Uso
<code>findAllByIdAsc</code>	Todos ordenados por id	Listado estable
<code>findByTituloContainingIgnoreCaseOrderByIdAsc</code>	Título contiene criterio	Búsqueda
<code>findById</code>	Clave primaria	Detalle
<code>count</code>	Cantidad de filas	Resumen
<code>sumarHoras</code>	SUM de horas	Resumen

Alcance controlado `JpaRepository` también ofrece `save` y `delete`, pero esta semana no se invocan. Su uso se incorporará con reglas, formularios y respuestas HTTP en la semana 10.

8 Mantener el DTO y agregar el mapper

CourseDto conserva exactamente el contrato de la semana 8. La clase Course es nueva y permanece dentro de persistencia; CourseMapper realiza la traducción explícita. Así, las plantillas y la API no dependen de anotaciones JPA.

src/main/java/pe/edu/utp/techlab/dto/CourseDto.java

```
package pe.edu.utp.techlab.dto;

public record CourseDto(long id, String titulo, int horas) {
}
```

src/main/java/pe/edu/utp/techlab/mapper/CourseMapper.java

```
package pe.edu.utp.techlab.mapper;

import org.springframework.stereotype.Component;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.entity.Course;

@Component
public class CourseMapper {

    public CourseDto toDto(Course course) {
        return new CourseDto(
            course.getId(),
            course.getTitulo(),
            course.getHoras()
        );
    }
}
```

Criterio de diseño

- La entidad representa almacenamiento; el DTO representa una necesidad de la aplicación.
- El mapper no consulta la base ni decide qué cursos mostrar.
- El controlador desconoce Hibernate y nunca recibe EntityManager.
- La vista no navega entidades ni activa consultas perezosas.

9 Sustituir el servicio en memoria

Reemplace por completo CourseService de la semana 8. Ya no existe una lista interna ni filtrado con streams sobre memoria. El servicio coordina repositorio y mapper dentro de una transacción de solo lectura.

src/main/java/pe/edu/utp/techlab/service/CourseService.java

```
package pe.edu.utp.techlab.service;

import java.util.List;
import java.util.Optional;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.entity.Course;
import pe.edu.utp.techlab.mapper.CourseMapper;
import pe.edu.utp.techlab.repository.CourseRepository;

@Service
@Transactional(readOnly = true)
public class CourseService {

    private final CourseRepository repository;
    private final CourseMapper mapper;

    public CourseService(
        CourseRepository repository,
        CourseMapper mapper) {
        this.repository = repository;
        this.mapper = mapper;
    }

    public List<CourseDto> buscar(String texto) {
        String criterio = Optional.ofNullable(texto)
            .orElse("")
            .strip();

        List<Course> entidades = criterio.isBlank()
            ? repository.findAllByOrderByByIdAsc()
            : repository
                .findByTituloContainingIgnoreCaseOrderByByIdAsc(criterio);

        return entidades.stream()
            .map(mapper::toDto)
            .toList();
    }

    public Optional<CourseDto> buscarPorId(long id) {
        return repository.findById(id)
            .map(mapper::toDto);
    }

    public long cantidad() {
        return repository.count();
    }

    public int totalHoras() {
        Long total = repository.sumarHoras();
        return Math.toIntExact(total == null ? 0L : total);
    }
}
```

Tabla 14 Decisiones del servicio

Decisión	Beneficio
Inyección por constructor	Dependencias explícitas y comprobables
@Transactional readOnly	Límite transaccional en la capa de aplicación
Consulta en repositorio	Filtrado ejecutado por la base de datos
Conversión antes de retornar	No escapan entidades administradas
Math.toIntExact	Detecta una suma fuera del rango int

Controladores y vistas

CourseController, CourseViewController, PortalController, las plantillas y el CSS permanecen sin cambios. Esa estabilidad es una prueba de diseño: se cambió la fuente de datos sin cambiar las rutas ni la representación.

Tabla 15 Contratos que no deben romperse

Ruta	Capa que cambió	Contrato que se conserva
GET /cursos	Servicio y repositorio	HTML 200 y búsqueda q
GET /cursos/{id}	Servicio y repositorio	Detalle 200 o 404
GET /cursos/resumen	Consultas agregadas	Cantidad y total de horas
GET /api/v1/cursos	Servicio y repositorio	Lista JSON
GET /api/v1/cursos/{id}	Servicio y repositorio	JSON 200 o 404

Revisión obligatoria Busque en todo el proyecto new CourseService(), List.of(new CourseDto y la anotación @Import(CourseService.class). Las dos primeras pertenecen a la versión en memoria; la tercera debe sustituirse en pruebas web por @MockitoBean.

10 Actualizar las pruebas

La persistencia se prueba contra MySQL real porque las migraciones, la intercalación y las restricciones son específicas del motor. Las pruebas web siguen siendo rápidas: aíslan los controladores y simulan CourseService.

src/test/java/pe/edu/utp/techlab/persistence/CoursePersistenceIntegrationTests.java

```
package pe.edu.utp.techlab.persistence;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.util.List;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.context.SpringBootTest;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.repository.CourseRepository;
import pe.edu.utp.techlab.service.CourseService;

@SpringBootTest
class CoursePersistenceIntegrationTests {

    @Autowired
    private CourseService service;

    @Autowired
    private CourseRepository repository;

    @Test
    void flywayCargaElCatalogoInicial() {
        assertEquals(3L, service.cantidad());
        assertEquals(48, service.totalHoras());

        List<String> titulos = service.buscar("").stream()
            .map(CourseDto::titulo)
            .toList();

        assertEquals(
            List.of("HTML y CSS", "Bootstrap", "Spring Boot"),
            titulos
        );
    }

    @Test
    void repositorioBuscaSinDistinguirMayusculas() {
        var resultado = repository
            .findByTituloContainingIgnoreCaseOrderByIdAsc("boot");

        assertEquals(1, resultado.size());
        assertEquals("Spring Boot", resultado.getFirst().getTitulo());
    }
}
```

Precondición MySQL debe estar healthy y DB_PASSWORD debe existir en la terminal antes de ejecutar esta prueba. @SpringBootTest inicia la aplicación, Flyway prepara el esquema y el test comprueba los datos a través de las capas reales.

src/test/java/pe/edu/utp/techlab/controller/CourseViewControllerTests.java parte 1 de 2

```

package pe.edu.utp.techlab.controller;

import static org.hamcrest.Matchers.containsString;
import static org.hamcrest.Matchers.hasSize;
import static org.mockito.BDDMockito.given;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.content;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.header;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.model;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.view;

import java.util.List;
import java.util.Optional;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.web.mvc.test.autoconfigure.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.service.CourseService;

@WebMvcTest({CourseViewController.class, PortalController.class})
class CourseViewControllerTests {

    @Autowired
    private MockMvc mvc;

    @MockitoBean
    private CourseService service;

    private CourseDto bootstrap;

    @BeforeEach
    void prepararDatos() {
        bootstrap = new CourseDto(2L, "Bootstrap", 16);
        given(service.buscar("")).willReturn(List.of(
            new CourseDto(1L, "HTML y CSS", 12),
            bootstrap,
            new CourseDto(3L, "Spring Boot", 20)
        ));
    }
}

```

src/test/java/pe/edu/utp/techlab/controller/CourseViewControllerTests.java parte 2 de 2

```

@Test
void listadoRenderizaModeloYHtml() throws Exception {
    mvc.perform(get("/cursos"))
        .andExpect(status().isOk())
        .andExpect(view().name("cursos/lista"))
        .andExpect(model().attribute("q", ""))
        .andExpect(model().attribute("cursos", hasSize(3)))
        .andExpect(content().string(
            containsString("Catálogo de cursos")
        ));
}

@Test
void filtroConservaConsultaYReduceResultados() throws Exception {
    var spring = new CourseDto(3L, "Spring Boot", 20);
    given(service.buscar("Spring")).willReturn(List.of(spring));

    mvc.perform(get("/cursos").param("q", "Spring"))
        .andExpect(status().isOk())
        .andExpect(model().attribute("q", "Spring"))
        .andExpect(model().attribute("cursos", hasSize(1)))
        .andExpect(content().string(containsString("Spring Boot")));
}

@Test
void detalleRenderizaElCursoSolicitado() throws Exception {
    given(service.buscarPorId(2L))
        .willReturn(Optional.of(bootstrap));

    mvc.perform(get("/cursos/2"))
        .andExpect(status().isOk())
        .andExpect(view().name("cursos/detalle"))
        .andExpect(model().attribute("curso", bootstrap))
        .andExpect(content().string(containsString("Bootstrap")));
}

@Test
void resumenMuestraCantidadYHoras() throws Exception {
    given(service.cantidad()).willReturn(3L);
    given(service.totalHoras()).willReturn(48);

    mvc.perform(get("/cursos/resumen"))
        .andExpect(status().isOk())
        .andExpect(model().attribute("cantidadCursos", 3L))
        .andExpect(model().attribute("totalHoras", 48));
}

@Test
void erroresConservanLosEstadosHttp() throws Exception {
    given(service.buscarPorId(999L)).willReturn(Optional.empty());

    mvc.perform(get("/cursos/999"))
        .andExpect(status().isNotFound());

    mvc.perform(get("/cursos").param("q", "x".repeat(61)))
        .andExpect(status().isBadRequest());
}

@Test
void portalRedirigeAlCatalogo() throws Exception {
    mvc.perform(get("/portal"))
        .andExpect(status().isFound())
        .andExpect(header().string("Location", "/cursos"));
}
}

```

@MockitoBean sustituye el servicio por un doble controlado dentro de la porción web. Cada prueba declara la respuesta necesaria; por eso no abre una conexión MySQL ni ejecuta Flyway. El controlador se evalúa por su contrato, no por la infraestructura de datos.

src/test/java/pe/edu/utp/techlab/controller/CourseControllerTests.java

```

package pe.edu.utp.techlab.controller;

import static org.mockito.BDDMockito.given;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.jsonPath;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import java.util.List;
import java.util.Optional;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.context.bean.override.mockito.MockitoBean;
import org.springframework.test.web.servlet.MockMvc;

import pe.edu.utp.techlab.dto.CourseDto;
import pe.edu.utp.techlab.service.CourseService;

@WebMvcTest(CourseController.class)
class CourseControllerTests {

    @Autowired
    private MockMvc mvc;

    @MockitoBean
    private CourseService service;

    @Test
    void apiListaYFiltroCursos() throws Exception {
        var spring = new CourseDto(3L, "Spring Boot", 20);
        given(service.buscar("boot")).willReturn(List.of(spring));

        mvc.perform(get("/api/v1/cursos").param("q", "boot"))
            .andExpect(status().isOk())
            .andExpect(jsonPath("$.id").value(3))
            .andExpect(jsonPath("$.titulo").value("Spring Boot"));
    }

    @Test
    void apiDevuelve404ParaIdInexistente() throws Exception {
        given(service.buscarPorId(999L)).willReturn(Optional.empty());

        mvc.perform(get("/api/v1/cursos/999"))
            .andExpect(status().isNotFound());
    }
}

```

Ejecutar la suite

Linux macOS o Git Bash

```

export DB_USER='techlab_app'
export DB_PASSWORD='valor_de_su_env'
./mvnw clean test
./mvnw clean package

```

Windows PowerShell

```

$env:DB_USER = 'techlab_app'
$env:DB_PASSWORD = 'valor_de_su_env'
.\mvnw.cmd clean test
.\mvnw.cmd clean package

```

Resultado esperado Maven termina con BUILD SUCCESS, cero fallos y cero errores. Si una prueba web intenta conectar a MySQL, revise que no conserve @Import(CourseService.class).

11 Verificar la aplicación completa

Ejecute el JAR empaquetado, no únicamente el botón del IDE. Esto comprueba que recursos, migraciones y dependencias forman parte de la entrega reproducible.

Terminal

```
java -jar target/techlab-web-0.0.1-SNAPSHOT.jar
```

Comprobar MySQL y Flyway

Abrir el cliente dentro del contenedor

```
docker compose exec mysql mysql -utechlab_app -p techlab
```

Ingrese la contraseña cuando el cliente la solicite. No escriba la clave como parte del comando porque quedaría en el historial de la terminal.

Consultas de verificación

```
SELECT id, titulo, horas
FROM cursos
ORDER BY id;

SELECT installed_rank, version, description, success
FROM flyway_schema_history
ORDER BY installed_rank;

EXIT;
```

Tabla 16 Matriz funcional de la semana 9

ID	Acción	Resultado esperado	Estado
P01	docker compose ps	mysql healthy	☐
P02	Consultar cursos	Tres filas y 48 horas	☐
P03	Consultar historial	V1 y V2 con success=1	☐
F01	Abrir /cursos	200 y tres tarjetas	☐
F02	Buscar q=boot	Solo Spring Boot	☐
F03	Abrir /cursos/2	Detalle de Bootstrap	☐
F04	Abrir /cursos/999	404 sin traza	☐
F05	Abrir /cursos/resumen	3 cursos y 48 horas	☐
A01	GET /api/v1/cursos	JSON con tres objetos	☐
A02	GET /actuador/health	status UP	☐
R01	Reiniciar aplicación	V1 y V2 no se repiten	☐
R02	Reiniciar contenedor	Datos aún disponibles	☐

Prueba de persistencia

1. Detenga la aplicación con Ctrl+C.
2. Ejecute `docker compose restart mysql` y espere el estado `healthy`.
3. Inicie nuevamente el JAR con las mismas variables de entorno.
4. Consulte `/cursos` y `flyway_schema_history`.
5. Confirme que los datos permanecen y que Flyway no vuelve a aplicar V1 ni V2.

Interpretación La permanencia proviene del volumen `techlab_mysql_data`. El historial permite a Flyway reconocer qué migraciones ya se ejecutaron y validar sus checksums.

12 Diagnosticar errores frecuentes

Tabla 17 Guía de diagnóstico

Síntoma	Causa probable	Comprobación y corrección
Access denied	Usuario o clave distintos	Compare <code>.env</code> y variables de la terminal
Communications link failure	Contenedor detenido o puerto incorrecto	Revise <code>ps logs</code> y 3307
Unknown database techlab	Volumen creado con otra configuración	Revise <code>MYSQL_DATABASE</code> y el volumen
Flyway checksum mismatch	Se editó una migración aplicada	Restaure el archivo y cree una nueva V3
Schema validation failed	Entidad y migración no coinciden	Compare nombres tipos <code>null</code> y longitud
No qualifying bean	Clase fuera del paquete raíz	Ubique <code>entity repository mapper</code> bajo <code>pe.edu.utp.techlab</code>
Application tests fail	<code>DB_PASSWORD</code> ausente	Defina la variable en la misma terminal
Prueba web busca DataSource	Se importó el servicio real	Use <code>@MockitoBean</code> y elimine <code>@Import</code>
Puerto 3307 ocupado	Otro MySQL está escuchando	Libere el puerto o documente un puerto alternativo

Reinicio controlado del laboratorio

Solo si el entorno local quedó inconsistente y no contiene información que deba conservarse, puede eliminar el volumen del laboratorio y comenzar desde cero.

Comando destructivo para el entorno local

```
docker compose down
docker volume rm techlab_mysql_data
docker compose up -d mysql
```

Advertencia La segunda orden elimina definitivamente los datos del volumen indicado. Verifique el nombre exacto `techlab_mysql_data` y no la ejecute en un entorno compartido o con datos que deban preservarse.

Calidad y seguridad

- No registre credenciales, cadenas con claves ni salidas de docker compose config.
- No habilite show-sql de forma permanente ni incluya parámetros sensibles en logs.
- No entregue target, datos del volumen, archivos .env ni configuraciones del IDE.
- No exponga entidades JPA directamente en JSON o plantillas.
- No edite migraciones ya aplicadas; agregue una nueva versión.
- No confíe solo en capturas; entregue comandos, pruebas y criterios observables.

13 Reto de aplicación

Evolucione el catálogo sin modificar V1 ni V2. El reto practica una migración incremental y mantiene el sistema en modo de lectura.

Requerimiento

1. Cree V3__add_categoria_to_cursos.sql.
2. Agregue categoria VARCHAR(40) NOT NULL con un valor inicial coherente para las filas existentes.
3. Actualice los tres cursos con categorías específicas dentro de la misma migración.
4. Mapee categoria en Course y agréguela a CourseDto y CourseMapper.
5. Muestre la categoría en listado detalle y JSON sin cambiar las rutas.
6. Agregue una consulta por categoría en CourseRepository y una prueba automática.
7. Ejecute la matriz de regresión y documente el historial V1 V2 V3.

Restricciones No elimine el volumen para aplicar V3, no altere V1 o V2, no use ddl-auto=update y no implemente todavía formularios de escritura.

Criterios del reto

Tabla 18 Condiciones de aprobación del reto

Criterio	Evidencia mínima
Migración incremental	V3 aplicada sobre la base existente
Compatibilidad	Aplicación inicia con ddl-auto=validate
Contrato ampliado	Categoría visible en HTML y JSON
Consulta	Filtro por categoría ejecutado por el repositorio
Regresión	Pruebas anteriores y nueva prueba aprobadas

Preguntas de cierre

1. Qué problema resuelve Flyway que ddl-auto=update no resuelve de forma auditable.
2. Por qué una entidad JPA no debería convertirse automáticamente en la respuesta REST.
3. Dónde termina la transacción cuando open-in-view está desactivado.
4. Qué prueba fallaría primero si se renombra la columna titulo solo en la entidad.
5. Qué partes de TechLab permanecerán estables cuando se agregue CRUD en la semana 10.

Referencias oficiales

- [Spring Boot 4.1.1 SQL Databases](#). Configuración de DataSource, JPA, Hibernate y repositorios.
- [Spring Boot Database Initialization](#). Flyway, orden de inicialización y estrategia única de esquema.
- [Spring Data JPA Reference](#). Repositorios, métodos derivados y consultas.
- [Hibernate ORM Documentation](#). Implementación JPA y mapeo objeto relacional.
- [MySQL 8.4 Release Notes](#). Línea LTS y versión 8.4.12.
- [Docker Official Image for MySQL](#). Variables y comportamiento de la imagen oficial.
- [Oracle Java SE Support Roadmap](#). Java 25 como versión LTS.